

Unix File System

Inodes, Directories, Block Addressing and TOC

Prashant Jadav

DEPSTAR – CHARUSAT

Learning Objectives

By the end of this lecture, students should be able to:

- Explain the organization of a Unix file system.
- Distinguish between files, directories, inodes and data blocks.
- Explain how a pathname is translated into an inode.
- Describe direct and indirect block addressing.
- Calculate the maximum file size using inode block pointers.
- Calculate the number of data blocks required for a file.
- Explain the role of inode block-address information.

Unix File System: Big Picture

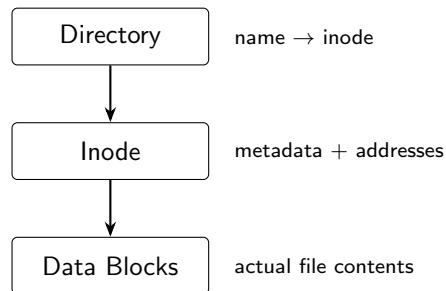
A Unix file system separates:

File metadata

and

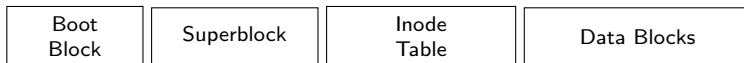
File data

- File type
- Permissions
- Owner and group
- File size
- Timestamps
- Link count
- Block addresses



Physical Organization of a Traditional Unix File System

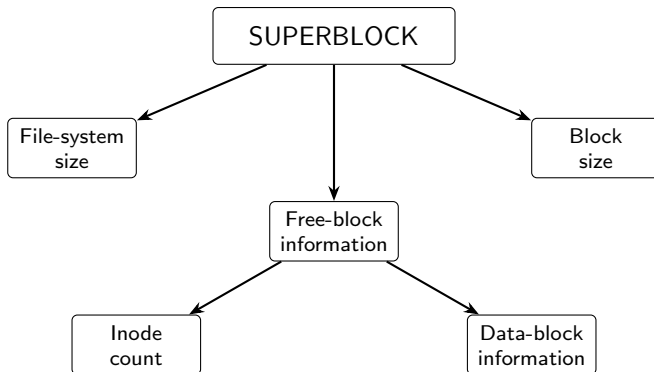
A simplified Unix disk can be organized as:



- **Boot Block:** contains boot-related information.
- **Superblock:** describes the file-system layout.
- **Inode Table:** contains file metadata and block addresses.
- **Data Blocks:** contain file and directory contents.

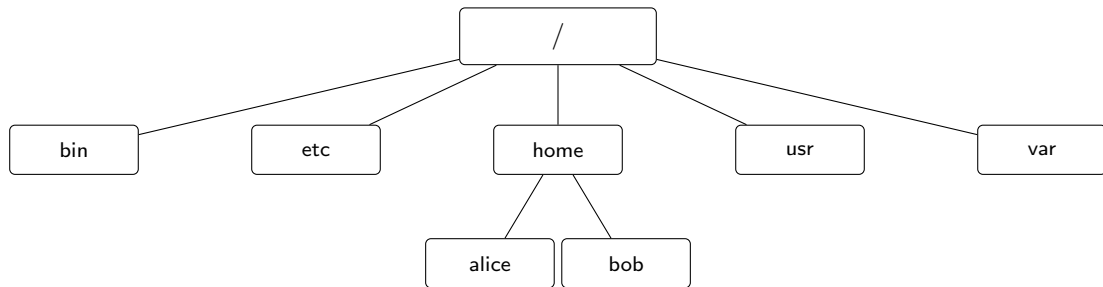
Role of the Superblock

The **superblock** contains information about the file system.



Unix Directory Structure

Unix organizes files using a hierarchical directory tree.

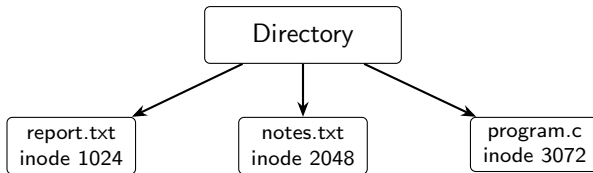


- ‘/’ is the root directory.
- Directories contain entries that associate names with inode numbers.
- A directory is itself a special type of file.

Directory Entry: Name to Inode

Consider:

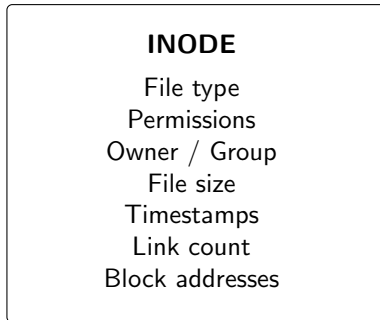
```
$ ls -i  
1024 report.txt  
2048 notes.txt  
3072 program.c
```



filename → inode number

What is an Inode?

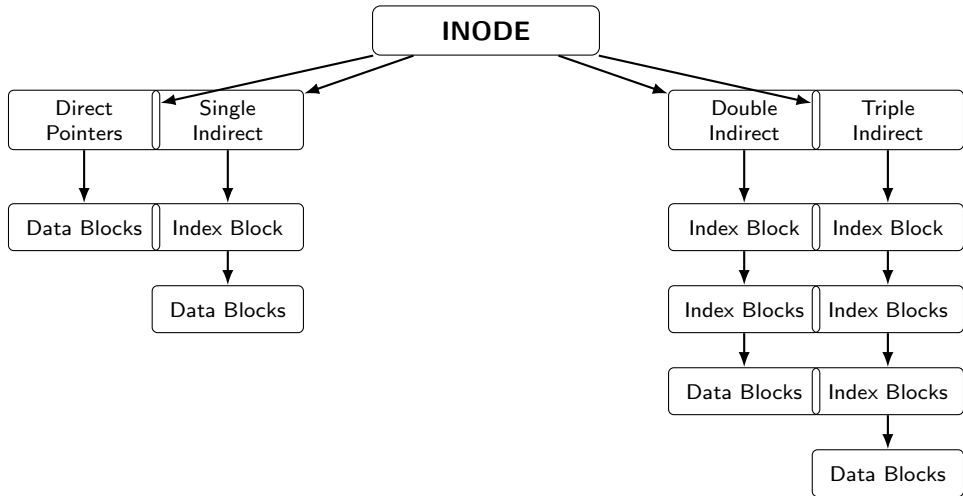
An **inode** is a data structure containing metadata about a file.



inode $\neq$ filename

The filename is associated with an inode through a directory entry.

Inode and Block Addressing



Inode and Block Addressing

- **Direct:** Inode $\rightarrow$ Data Block
- **Single indirect:** Inode $\rightarrow$ Index Block $\rightarrow$ Data Blocks
- **Double indirect:** Inode $\rightarrow$ Index $\rightarrow$ Index $\rightarrow$ Data Blocks
- **Triple indirect:** Inode $\rightarrow$ Index $\rightarrow$ Index $\rightarrow$ Index $\rightarrow$ Data Blocks

Direct Block Addressing

A direct pointer points directly to a data block.



Suppose:

$$B = 4 \text{ KB}$$

and there are 12 direct pointers.

$$12 \times 4 = 48 \text{ KB}$$

Direct capacity = 48 KB

Single Indirect Addressing

A single-indirect pointer points to an index block.



Suppose:

$$B = 4096 \text{ bytes}, \quad P = 4 \text{ bytes}$$

$$N = \frac{4096}{4} = 1024$$

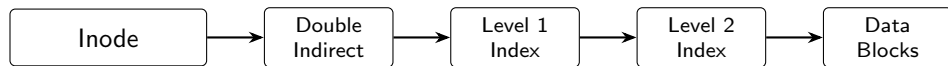
Therefore:

$$1024 \times 4 \text{ KB} = 4096 \text{ KB} = 4 \text{ MB}$$

Single-indirect capacity = 4 MB

Double Indirect Addressing

A double-indirect pointer uses two levels of index blocks.



If:

$$N = 1024$$

then:

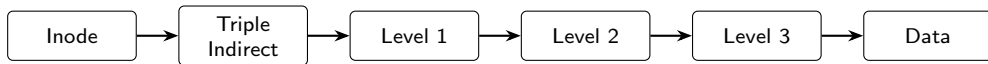
$$N^2 = 1024^2 = 1,048,576$$

With 4 KB blocks:

$$1,048,576 \times 4 \text{ KB} = 4 \text{ GB}$$

Triple Indirect Addressing

Triple indirect adds a third level of indexing.



With:

$$N = 1024$$

$$N^3 = 1024^3 = 1,073,741,824$$

For 4 KB blocks:

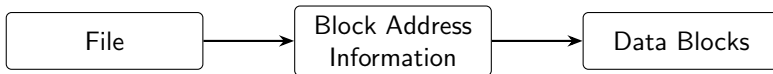
$$N^3 \times 4 \text{ KB} \approx 4 \text{ TB}$$

TOC: Teaching Interpretation

Some textbooks or classroom models use the term:

TOC – Table of Contents

as an analogy for the block-address information associated with a file.



The idea is:

Which disk blocks contain my file?

Terminology

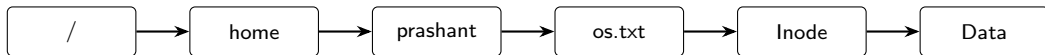
In standard Unix terminology, the important structure is the **inode and its block pointers**. “TOC” is a teaching abstraction, not a universal Unix data structure name.

Pathname Resolution

Consider:

```
/home/prashant/notes/os.txt
```

The kernel resolves the path component by component.



Pathname → Directory → Inode → Blocks → Data

Worked Example: Locating a File

Suppose a directory contains:

Name	Inode Number
student	120
a.txt	450
program.c	451

For:

/home/student/a.txt

the kernel eventually finds:

a.txt $\rightarrow$ *inode* 450

Suppose inode 450 contains direct block addresses:

100, 105, 109, 115

The blocks do not have to be contiguous.

Worked Calculation: Addresses per Index Block

Given:

$$B = 4 \text{ KB}$$

$$P = 4 \text{ bytes}$$

First:

$$4 \text{ KB} = 4096 \text{ bytes}$$

Therefore:

$$N = \frac{4096}{4} = 1024$$

Hence:

1024 addresses/index block

Therefore:

$$\text{Single indirect} = 1024$$

$$\text{Double indirect} = 1024^2$$

$$\text{Triple indirect} = 1024^3$$

Worked Calculation: Maximum File Size

$$D = 12 \quad (\text{Direct Pointers})$$

$$N = 1024 \quad (\text{addresses per index block})$$

$$B = 4 \text{ KB} \quad (\text{disk block size})$$

Maximum number of data blocks:

$$D + N + N^2 + N^3$$

$$= 12 + 1024 + 1024^2 + 1024^3$$

$$= 1,074,791,436$$

Worked Calculation: Maximum File Size

Maximum file size:

$$1,074,791,436 \times 4 \text{ KB}$$

$$= 4,299,165,744 \text{ KB}$$

Approximately:

4.0 TB

Worked Problem: 50 KB File

Given:

$$B = 4 \text{ KB}$$

and:

$$D = 12$$

direct pointers. and File size:

$$50 \text{ KB}$$

Number of required data blocks:

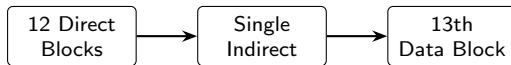
$$\left\lceil \frac{50}{4} \right\rceil = 13$$

The first 12 blocks use direct pointers. Therefore additional block required:

$$13 - 12 = 1$$

Single-indirect addressing is required

Worked Problem: 50 KB File



Worked Problem: 10 MB File

Given:

$$B = 4 \text{ KB}$$

Number of data blocks required:

$$\frac{10 \times 1024}{4} = 2560 \text{ data blocks}$$

Step 1: Direct Blocks

The inode provides 12 direct pointers.

$$2560 - 12 = 2548$$

So, after using the direct pointers:

2548 data blocks remain

Worked Problem: 10 MB File

$$B = 4 \text{ KB}$$

$$\text{Data blocks} = \frac{10 \times 1024}{4} = 2560$$

$$\underbrace{2560}_{\text{Total}} - \underbrace{12}_{\text{Direct}} = 2548$$

$$\underbrace{2548}_{\text{Remaining}} - \underbrace{1024}_{\text{Single Indirect}} = 1524$$

$12 \text{ Direct} + 1024 \text{ Single Indirect} + 1524 \text{ Remaining}$

Worked Problem: Double-Indirect Addressing

Step 3: Double Indirect

The remaining 1524 data blocks are addressed through the double-indirect pointer.

A double-indirect pointer can address:

$$1024 \times 1024 = 1,048,576 \text{ data blocks}$$

Therefore, 1524 data blocks easily fit within the double-indirect addressing capacity.

Final distribution of data blocks:

12	→	Direct
1024	→	Single indirect
1524	→	Double indirect

$$12 + 1024 + 1524 = 2560 \text{ data blocks}$$

Why Multiple Levels of Indirection?

Without indirect addressing:

Large file $\Rightarrow$ Large inode

With hierarchical addressing:

Small inode + Index blocks $\Rightarrow$ Large file



The design trades:

additional indirection

for:

much larger files

Important Formulae

Number of addresses in one index block:

$$N = \frac{\text{Block Size}}{\text{Pointer Size}}$$

Addressable data blocks:

$$\begin{aligned}\text{Direct} &= D \\ \text{Single} &= N \\ \text{Double} &= N^2 \\ \text{Triple} &= N^3\end{aligned}$$

Maximum data blocks:

$$D + N + N^2 + N^3$$

Maximum file size:

$$(D + N + N^2 + N^3) \times B$$

Problem 1

A Unix-like file system has:

$$B = 8 \text{ KB}$$

and:

$$P = 4 \text{ bytes}$$

An inode has 10 direct pointers.

Find:

- 1 Number of addresses in one indirect block.
- 2 Number of blocks addressable using double indirection.
- 3 Capacity provided by the direct pointers.

Problem 2

A file is:

70 KB

with block size:

4 KB

and 12 direct pointers.

Determine whether single-indirect addressing is required.

Key Takeaways

- ➊ A directory maps names to inode numbers.
- ➋ An inode stores metadata and block-address information.
- ➌ File data is stored in data blocks.
- ➍ Direct pointers provide efficient access to small files.
- ➎ Indirect pointers allow large files without huge inodes.
- ➏ One index block contains:

$$\frac{\text{block size}}{\text{pointer size}}$$

addresses.

- ➐ Multi-level indexing gives:

$$N, N^2, N^3$$

addressable blocks.

Complete Unix File-System View

